

CSL214: Data Structure and Program Design – II

Lab Assignment (Announcement: 10th April, 2026)

Due Date: 20th April, 2026 (By 12:00 noon)

For batch - R1

Notes

1. You can choose to do the assignment alone or in a group of 2 students. If done in a group, both students should be from the same practical-batch.
 2. No late submission will be allowed.
 3. No copying/sharing the code across the groups. If found copied, NO marks will be awarded to the assignment for all such groups including original authors.
 4. For this assignment, Tree data structure is required to be used. Hence, choose Trees (either AVL tree OR B-tree OR B+tree) for any storage requirement you may have. Use of file operations to store/retrieve persistent data, would be given more credits.
 5. Evaluation will be done during 20th April to 22nd April 2026. Please make sure that you attend your lab hours during this week. For some batches/students, the evaluation may take place at scheduled time other than lab hours as will be announced.
 6. You need to submit your source files by
 - Attaching these files in a mail by sending the mail to dspd.assignment@gmail.com
Please attach only *.c, *.h files (and other i/p text files if any). NO *.objs and *.exe to be attached.
 7. Please mark your subject line as *DSPD-2-Assignment: Your Enrollment numbers.*
-

Music Playlist Management System

Design and implement a **music playlist management system similar to Spotify** using *either an AVL tree or B-tree or B+ tree* as a data structure. The system should allow users to **create and manage a playlist dynamically** where each node/record in the tree represents a song containing details such as **SongID, Song Title, and Artist Name**. The playlist should support efficient navigation between songs using. Each playlist is by default, should be easily sorted based on SongID.

The program should provide a **menu-driven interface** where users can perform playlist operations and simulate a music player. The application must provide functionality to **add songs, delete songs, display the playlist, play the next song, play the previous song, and search for songs**. Additionally, the system should simulate basic features of modern music players such as **shuffle mode, repeat playlist, and play history**.

1. **Data Structure : Playlist** : Each node represents a **song**.

Node Structure

Song Node

SongID

Title

Artist

Functionalities to be implemented :

- i. **Add Song** – Insert a new song into the playlist. Keep the playlist sorted on **SongID**.
- ii. **Delete Song** – Remove a song using its **SongID**.
- iii. **Display Playlist** – Show all songs in the playlist.
 - a. **Display sorted based on Title**
 - b. **Display sorted based on Artist**
 - c. **Display sorted first on Artist, and for each artist based on Title.**
- iv. **Search Song** – Find a song by
 - a. SongID
 - b. Title
 - c. Artist
- v. **Shuffle Mode** – Change the database of songs/nodes in some random order.
- vi. **Repeat Mode** – Repeat/display the playlist continuously.
- vii. **Range-Search** – Given two SongIDs s_1 and s_2 wherein $s_1 < s_2$ in terms of the corresponding order (based on type, either integer ‘less-than’ or lexicographic ‘less-than’ as the case maybe), list down all the songs having SongIDs “in-between” s_1 and s_2 (both inclusive).

2. Database: Play History: Used to store songs already played.

Functionalities :

- i. Add played song to the history.
- ii. Display history in the chronological order
- iii. Display history in the reverse chronological order.
- iv. Play History : When a song is played insert it in Play History

3. Operations on multiple playlists

Assume that each song playlist/database is sorted based on SongID. Consider two such playlists are available. Write functions for the following operations. Note that two playlists may have some songs in common.

- i. **Union of playlists** – Create a third playlist which is a *union* of two playlists, that is, it should contain all songs from both the databases sorted based on SongID
- ii. **Intersection of playlists** - Create a third playlist which is an *intersection* of two playlists, that is, it should contain songs common to both the playlists and that is sorted based on SongID
- iii. **Difference of playlists** - Create a third playlist which is *difference* of two playlists, that is, it should contain songs from first playlist that are not present in the second playlist.
- iv. **Symmetric difference of playlists** - Create a third playlist which is a *symmetric difference* of two playlists, that is, it should contain songs from first playlist that are not present in the second playlist and also the songs which are present in the second playlist but not the first.

CSL214: Data Structure and Program Design – II

Lab Assignment (Announcement: 10th April, 2026)

Due Date: 20th April, 2026 (By 12:00 noon)

For batch – R2

Notes

1. You can choose to do the assignment alone or in a group of 2 students. If done in a group, both students should be from the same practical-batch.
 2. No late submission will be allowed.
 3. No copying/sharing the code across the groups. If found copied, NO marks will be awarded to the assignment for all such groups including original authors.
 4. For this assignment, Tree data structure is required to be used. Hence, choose Trees (either AVL tree OR B-tree OR B+tree) for any storage requirement you may have. Use of file operations to store/retrieve persistent data, would be given more credits.
 5. Evaluation will be done during 20th April to 22nd April 2026. Please make sure that you attend your lab hours during this week. For some batches/students, the evaluation may take place at scheduled time other than lab hours as will be announced.
 6. You need to submit your source files by
 - Attaching these files in a mail by sending the mail to dspd.assignment@gmail.com
Please attach only *.c, *.h files (and other i/p text files if any). NO *.objs and *.exe to be attached.
 7. Please mark your subject line as *DSPD-2-Assignment: Your Enrollment numbers.*
-

Railway Reservation System

Implement a railway reservation system. Maintain a database of coaches. Each coach has a type like 1AC, 2AC, 3AC, Sleeper. There are additionally the engine and the pantry.

- 1AC has capacity 16. The numbers 001, 002, 003... can be used as coach numbers of 1AC.
 - 2AC has capacity 48 : The numbers 101, 102, 103... can be used as coach numbers for 2AC.
 - 3AC has capacity 64 : The numbers 201, 202, 203... can be used as coach numbers for 3AC.
 - Sleeper has capacity 72 : The numbers 301, 302, 303... can be used as coach numbers for Sleeper.
 - The sequence of coaches is *engine, 1st AC, Pantry, 2nd AC, 3rd AC, Sleeper.*
- Each coach maintains database of seat numbers which are marked as *unavailable* or *available*. You are required to use trees to maintain this database.
 - Each seat has properties like *seat no, coach type, berth type*. Berth types are L, M, U, SL, SU.
 - Each passenger has *name, DoB, gender, age, coach-number, seat-no*
 - The list of all passengers is maintained as a separate database.

Implement the following operations:

1. When a passenger wants to do a reservation, she mentions which type of coach she wants and number of seats as well as types of seats. If available, the seats are booked. If they are *not available* message is displayed.
 - i. Implement a functionality of booking only if all seat-type preferences are available in the same coach.

- ii. There could be multiple coaches of the same type, it may search in all before mentioning that seat is unavailable. If they are not available in the same coach, seats are booked from multiple coaches so that seat-type preference is maintained.
 - iii. If all seat-types are not available, do the booking such that maximum number of seat-type preferences are booked.
 - iv. If seats are not available, waiting-list number may be given to reservation request.
- 2. Sort the database of passengers according to coach number and for same coach number according to seat-number.
- 3. Sort the database of passengers who belong to a particular coach by passenger-name alphabetically.
- 4. Display Functions
 - i. Display all passengers sorted by names
 - ii. Display all passengers sorted by coach-numbers
 - iii. Display all passengers who were booked lower-berth seats (L or SL) along with their ages.
 - iv. Display all passengers (along with their ages) who are senior citizens (age > 60 yrs) were **not** booked lower-berth seats (L or SL).
- 5. Display number of available seats of each type of coach.
- 6. Suppose a passenger cancels the reservation, her and other passenger names in the same reservation request should be removed from the passenger database. These seat numbers should be marked as available and the waitlist can be advanced. You may allow partial cancellation for a ticket as well. (That is, not all passengers on a ticket are cancelled).
- 7. Range-Search : Given names N1 and N2 of two passengers P1 and P2, list down details of all passengers whose names are “in-between” the names N1 and N2 (both inclusive) as per the lexicographic order.

CSL214: Data Structure and Program Design – II
Lab Assignment (Announcement: 10th April, 2026)
Due Date: 20th April, 2026 (By 12:00 noon)

For batch – R3

Notes

1. You can choose to do the assignment alone or in a group of 2 students. If done in a group, both students should be from the same practical-batch.
 2. No late submission will be allowed.
 3. No copying/sharing the code across the groups. If found copied, NO marks will be awarded to the assignment for all such groups including original authors.
 4. For this assignment, Tree data structure is required to be used. Hence, choose Trees (either AVL tree OR B-tree OR B+tree) for any storage requirement you may have. Use of file operations to store/retrieve persistent data, would be given more credits.
 5. Evaluation will be done during 20th April to 22nd April 2026. Please make sure that you attend your lab hours during this week. For some batches/students, the evaluation may take place at scheduled time other than lab hours as will be announced.
 6. You need to submit your source files by
 - Attaching these files in a mail by sending the mail to dspd.assignment@gmail.com
Please attach only *.c, *.h files (and other i/p text files if any). NO *.objs and *.exe to be attached.
 7. Please mark your subject line as *DSPD-2-Assignment: Your Enrollment numbers.*
-

Ride-Hailing System for Cabs and Bikes.

Consider a ride-hailing agency that provides both cabs and bikes on rent for dropping passengers. When a passenger requires a ride, he/she sends a request through the mobile app, optionally specifying his preferred vehicle type (cab or bike) or leaving it as “any”. As soon as a request is generated, the current location of the passenger is tracked and the system searches for all nearby free vehicles (cabs and bikes) of the preferred type (or any if not specified). The nearest free vehicle is booked for the passenger. The system must maintain complete information about drivers, passengers, booking history, and real-time status of all vehicles.

You are required to implement this system using trees (AVL or B-tree or B+-tree) in C language. All data must be stored dynamically using structures and pointers. The system must support at least the functionalities listed below.

Data Structures:

You must create the following databases:

1. Driver Database

Stores details of all drivers (both cab and bike drivers) working for the agency. Each node contains:

- d_ID (unique integer), name (string), vehicle_type (integer: 0 for cab, 1 for bike), current_location (structure with x and y coordinates, integers), status (integer: 0 = free, 1 = booked), total_earnings (float) – can be updated after each ride, next pointer

2. Passenger Database

Stores details of all passengers who have ever booked a ride. Each node contains:

- p_ID (unique integer), name (string), mobile_no (string), frequency (integer) – total number of rides taken by this passenger, next pointer

3. Booking History Database

Stores every completed ride. Each node contains:

- booking_id (unique integer), d_ID (driver ID), p_ID (passenger ID), vehicle_type (0/1) – useful for quick analysis, distance_travelled (float, in km), fare (float, calculated as distance × rate), date (optional, can be simulated by an integer timestamp), next pointer

Functions to Implement:

You must implement following functions. You can add more if needed.

1. void addDriver(Driver **head, int id, char name[], int type, int x, int y)

Adds a new driver at the end of the Driver Database. The driver is initially free (status = 0) and total_earnings = 0. Coordinates are given as input.

2. void addPassenger(Passenger **head, int id, char name[], char mobile[])

Adds a new passenger in the passenger database with frequency = 0. If a passenger with the same id or mobile already exists, do not add a duplicate (you may simply return or update nothing).

3. Driver* findNearestVehicle(Driver *dHead, int p_x, int p_y, int prefType)

Finds the nearest free vehicle (status 0) of type prefType (0 for cab, 1 for bike, -1 for any) within a 5 km radius from the passenger's location (p_x, p_y). Use Euclidean distance:

$$\text{distance} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}.$$

Return a pointer to that driver node. If no vehicle is available within 5 km, return NULL.

Note: For simplicity, assume coordinates are in kilometres and no conversion is needed.

4. int requestRide(Driver *dHead, Passenger *pHead, Booking **bHead, int p_id, int p_x, int p_y, int prefType)

Handles a new ride request.

- Verify that the passenger exists (by p_id) in the Passenger database. If not, print an error and return -1.
- Call findNearestVehicle() to obtain the nearest free driver of the preferred type.
- If a driver is found, create a new booking node with a unique booking_id (you can maintain a static counter). Set distance_travelled = 0 and fare = 0 initially.
- Change the driver's status to 1 (booked).

- Append the booking to the Booking History list.
- Return the new booking_id.
- If no driver is found, return -1.

5. **void completeRide(Driver *dHead, Passenger *pHead, Booking *bHead, int booking_id, float distance)**

Completes a ride.

- Find the booking with the given booking_id in the Booking list.
- Update its distance_travelled with the provided distance.
- Calculate fare based on vehicle type:
 - Cab (type 0) : ₹10 per km
 - Bike (type 1) : ₹5 per km
- Add this fare to the driver's total_earnings (update the driver node).
- Increment the passenger's frequency by 1.
- Set the driver's status back to 0 (free).
- (Optionally, you may also store the completion time.)

6. **float calculateDriverEarnings(Driver *dHead, int d_id)**

Computes the total earnings of a specific driver by summing the fare of all bookings associated with that driver in the Booking database.

Alternatively, you could use the total_earnings field if you keep it updated – but this function demonstrates traversal and summation.

7. **void displayTopDrivers(Driver *dHead, Booking *bHead)**

Displays the names of the three most successful drivers based on their total earnings.

- Compute earnings for each driver (using calculateDriverEarnings or the stored field).
- Sort drivers by earnings (or find top three).
- Print their names and total earnings.

8. **void displayFrequentPairs(Driver *dHead, Passenger *pHead, Booking *bHead)**

Finds the passenger-driver pair (i.e., a specific (d_ID, p_ID)) that has travelled together the most number of times.

- Count occurrences of each pair from the Booking database.
- Retrieve the names of the driver and passenger from the respective databases.
- Display their names and the number of rides they shared.

9. **void displayAvailableVehicles(Driver *dHead)**

Prints the details (ID, name, vehicle type, current location) of all drivers whose status is free (0).

- Clearly indicate whether it's a cab or a bike.

10. void updateDriverLocation(Driver *dHead, int d_id, int new_x, int new_y)

Updates the current coordinates of a driver (simulating movement).

- Find the driver by d_id and set his current_location to (new_x, new_y).
- If the driver is booked, you may optionally prevent location update or allow it – specify your choice.

11. void deleteDriver(Driver **dHead, int d_id)

Removes a driver from the Driver database when they leave the agency.

- Ensure the driver is currently free (status 0). If booked, print an error and do not delete.
- Free the node and adjust links.

12. void displayBookingHistory(Booking *bHead)

Prints all bookings with details: booking ID, driver ID, passenger ID, vehicle type, distance, fare.

13. Range-Search – Given two passengers with P_ID1 and P_ID2 as their passenger IDs, print details of all passengers having their passenger IDs “in between” P_ID1 and P_ID2 (both inclusive).

Additional Notes

- Use appropriate typedef for structures.
- For simplicity, assume coordinates are integers representing kilometres. The 5 km radius check must use Euclidean distance (you may use sqrt from math.h; remember to link with -lm).
- The fare rates are fixed: Cab = ₹10/km, Bike = ₹5/km.
- All functions must handle edge cases gracefully (e.g., empty databases, non-existent IDs, no vehicle found, etc.) and print appropriate messages.
- You are free to design additional helper functions (e.g., for sorting, searching) but the above 12 (or at least 10) must be present and correctly implemented.
- The program should provide a menu-driven interface to test all functionalities. The user should be able to:
 - Add a driver
 - Add a passenger
 - Request a ride
 - Complete a ride
 - Display top drivers
 - Display frequent pairs
 - Display available vehicles
 - Update driver location
 - Delete a driver
 - Display booking history

- Exit

CSL214: Data Structure and Program Design – II

Lab Assignment (Announcement: 10th April, 2026)

Due Date: 20th April, 2026 (By 12:00 noon)

For batch – R4

Notes

1. You can choose to do the assignment alone or in a group of 2 students. If done in a group, both students should be from the same practical-batch.
 2. No late submission will be allowed.
 3. No copying/sharing the code across the groups. If found copied, NO marks will be awarded to the assignment for all such groups including original authors.
 4. For this assignment, Tree data structure is required to be used. Hence, choose Trees (either AVL tree OR B-tree OR B+tree) for any storage requirement you may have. Use of file operations to store/retrieve persistent data, would be given more credits.
 5. Evaluation will be done during 20th April to 22nd April 2026. Please make sure that you attend your lab hours during this week. For some batches/students, the evaluation may take place at scheduled time other than lab hours as will be announced.
 6. You need to submit your source files by
 - Attaching these files in a mail by sending the mail to dspd.assignment@gmail.com
Please attach only *.c, *.h files (and other i/p text files if any). NO *.objs and *.exe to be attached.
 7. Please mark your subject line as *DSPD-2-Assignment: Your Enrollment numbers.*
-

Thin Inventory-led Delivery Model (Blinkit)

There are 3 important aspects of the *Blinkit* model, the customers, the delivery-riders and the *dark-stores* (or inventories) spread across the city.

Implement the databases for the following using trees (either AVL tree or B-Tree or B+-Tree).

1. **Customers-database** – Database of registered customers and their data like Name, Aadhar, Address, contact details (including address with PinCode), Blinkit account ID, Account Balance. Two more internal databases are to be maintained for each customer: database-of-completed-orders and database-of-pending-orders.
2. **Delivery-riders-database** – Name, ID, contact number, location, dark-store-id (to which assigned to), Monthly income for last 12 months. A delivery-boy earns 10% on each order he/she completes.
3. **Dark-store-database** – Database of dark stores in a city. Each node should contain Dark-store-ID, location, Inventory details (another database that stores list of items and number of each item available).

Notes

1. Assume that every PinCode has at-max 1 dark-store. Some PinCodes may not have any dark-store.
2. Assume that all addresses of customers, delivery riders and dark stores are having PinCodes. Also, assume that there is a distance table that has stored distance of each PinCode to every other PinCode.

Decide how do you design such a table.

Following operations are to be provided

1. Registration of a customer
2. Update data for customer
3. Deletion of a customer
4. Display data of customers
 - a. In the descending order based on expenses on past purchases
 - b. Based on PinCode of the customers
5. Registration of a delivery rider : A delivery rider is assigned to the dark store closest to his residence
6. Deletion of delivery-rider.
7. Display the data of delivery-riders
 - a. In the descending order based on earning of delivery riders in the whole city for last 3 months
 - b. In the descending order of the number of orders completed by the delivery-rider in last 1 month in the whole city
 - c. Based on PinCode first, and within a PinCode in the descending order based on the earning of delivery riders in that PinCode
8. **Placing an order** – A customer places an order that takes ItemID and number-of-items as input. The system finds the nearest dark-store from where the complete order can get fulfilled. Once customer is assigned to a dark-store, available delivery-rider attached to the same store is identified. (As time is of essence, if there is no delivery-rider for the dark store who is free, an available delivery-rider from the adjacent/neraby dark-store may get identified.) The order is attached to the identified delivery-rider (who is now *not-available*), (you may want to keep a separate database for orders at a dark-store?). Time of placing the order may also get stored. The item and number of item gets deleted from the inventory availability.
9. **Delivery of the order** : Delivery-rider delivers the order to the customer who pays the amount. The dark-store marks the order as completed. The delivery-rider gets 10% of the amount deposited in his account, and list of orders completed for him/her is incremented. The order is moved from pending order to completed order for all the three : customer, delivery-rider and the dark-store.
10. **10. Range-Search** – Given Blinkit account IDs B1 and B2 of two customers, list down / print all the details of all customers having their Blinkit IDs “in-between” B1 and B2 (both inclusive).

CSL214: Data Structure and Program Design – II

Lab Assignment (Announcement: 10th April, 2026)

Due Date: 20th April, 2026 (By 12:00 noon)

For batch – R5

Notes

1. You can choose to do the assignment alone or in a group of 2 students. If done in a group, both students should be from the same practical-batch.
 2. No late submission will be allowed.
 3. No copying/sharing the code across the groups. If found copied, NO marks will be awarded to the assignment for all such groups including original authors.
 4. For this assignment, Tree data structure is required to be used. Hence, choose Trees (either AVL tree OR B-tree OR B+tree) for any storage requirement you may have. Use of file operations to store/retrieve persistent data, would be given more credits.
 5. Evaluation will be done during 20th April to 22nd April 2026. Please make sure that you attend your lab hours during this week. For some batches/students, the evaluation may take place at scheduled time other than lab hours as will be announced.
 6. You need to submit your source files by
 - Attaching these files in a mail by sending the mail to dspd.assignment@gmail.com
Please attach only *.c, *.h files (and other i/p text files if any). NO *.objs and *.exe to be attached.
 7. Please mark your subject line as *DSPD-2-Assignment: Your Enrollment numbers.*
-

Travel Planner

Design and implement Travel Planner application that combines features of trip booking (like MakeMyTrip) and navigation guidance (like Google Maps).

The application should allow a user to:

- A. Plan a trip from a **starting location to a final destination**
- B. Store a sequence of **travel activities** such as flights, hotel stays, and tourist visits
- C. Provide **navigation directions** to reach each location
- D. Dynamically **insert, delete, or update trip steps and navigation directions**

The system should represent both **trip activities and navigation directions using trees (either AVL or B-Tree or B+-Tree).**

- The **Trip Database** stores the sequence of travel activities.
- Each **Trip Node contains a Navigation Database** that stores checkpoints or directions required to reach that activity.

The application uses at-least two databases.

1. Trip Database : Stores major travel activities.

Each node represents the following:

- Flight details

- Hotel stay details
- Tourist location details
- Transportation details
- **Trip Node**

Arrange the database sorted on chronological order (i.e. ordered in time).

2. Navigation Database (Inside Each Step)

Each travel activity may have a list of **navigation directions** (E.g. Walk → Turn Left → Metro → Destination) required to reach to that place from the previous place (i.e. “previous” Trip Node in the Trip database).

Node for Navigation Steps

Field	Description
Direction	Navigation step
distance	Distance to next step
next	Pointer to next instruction

Operations to be supported

A. Trip Level Operations

1. **Create trip** : Initialize a new trip itinerary.
2. **Add travel activity** : Add a new activity “at the end” of the itinerary.
3. **Insert Activity** : Insert a travel activity between two existing activities appropriately based on the travel-date.
4. **Delete Activity** : Remove an activity if the plan changes.
5. **Display Trip Plan** : Traverse the trip database and show the entire itinerary.

B. Navigation Operations (Directions / Checkpoints)

1. **Create Route**
2. **Add Direction** : Add a navigation step at appropriate place.
3. **Insert Direction (Detour / Checkpoint** : Insert a navigation instruction between two navigation steps.
Hint : Decide whether you need to give stepID to each step.
4. **Delete Direction** : Remove a navigation step if the route changes.
5. **Search Direction** : Check whether a specific navigation instruction exists between a source to a target.
6. **Update Direction**

- **Display Navigation Route** : Traverse and display navigation checkpoints.

Important New Operation

Range-Search – Given two dates D1 and D2, create a list and display all the trip details for the tourist during the dates D1 and D2 (both inclusive).

C. Other Operations

1. Given a source location and a destination location, find out if there exists a path from source to destination. If yes, print the travel plan giving all the travel details (stay/transport/directions etc)
2. Detect if in the travel plan, if you are reaching/staying at the same location multiple number of times. Print all such locations visited multiple times, and the number of times they are visited.
3. Sort the trip dataabase in the descending order of hotel charges at different locations.